EMBEDDED SYSTEMS PROJECT

Snake Game on AVR Microcontroller

ATmega32 · 20×4 LCD · AVR-Embedded C

STUDENT 01 · Reem Eion
STUDENT 02 · Fatma Eslam
STUDENT 03 · Malak Mamdouh
STUDENT 04 · Merna Mohamed

System Architecture

Software Architecture (Layered Design)

APPLICATION LAYER
main.c · FUNCTION.c

HAL LAYER
LCD · Buzzer · Segment

MCAL LAYER
DIO

LIB LAYER
STD_TYPES · BIT_MATH · FUNCTION.h

BCD 7-Segment

0-99 · BCD OUTPUT

```c
void SevenSegment_Init(void) {

    MDIO_voidSetPortDirection(SEVEN_SEGMENT_PORT, 0xFF);
}

void SevenSegment_DisplayNumber(u8 NumberOfScore) {
    if (NumberOfScore <= 99) {
        u8 Units = NumberOfScore % 10;
        u8 Tens  = NumberOfScore / 10;

        u8 Score_Count = (Tens << 4) | Units;

        MDIO_voidSetPortValue(SEVEN_SEGMENT_PORT, Score_Count);
    }
}
```

BUZZER

```
void HBuzzer_voidOn(void)
{
    MDIO_voidSetPinValue(
            BUZZER_PORT,
            BUZZER_PIN,
            DIO_HIGH);
}
```

```
void HBuzzer_voidOff(void)
{
    MDIO_voidSetPinValue(
            BUZZER_PORT,
            BUZZER_PIN,
            DIO_LOW);
}
```

```
void HBuzzer_voidInit(void)
{
    MDIO_voidSetPinDirection(
            BUZZER_PORT,
            BUZZER_PIN,
            DIO_OUTPUT);

    HBuzzer_voidOff();
}
```

```
void HBuzzer_voidBeep(void)
{
    HBuzzer_voidOn();

    _delay_ms(100);

    HBuzzer_voidOff();
}
```

LCD

```c
void HLCD_voidSendCommand(u8 A_u8Command);
void HLCD_voidSendData(u8 A_u8Command);
void HLCD_voidInit(void);
void HLCD_voidSendString(u8 *A_Pu8String);
void HLCD_voidClearDisplay(void);
void HLCD_voidDisplayNumberUnsigned (u32 A_u32Number);
void HLCD_voidDisplayNumberSigned (s32 A_s32Number);
void HLCD_voidGoToPos (LCD_ROWS A_LcdRowNo,LCD_COLS A_LcdColNo);
void HLCD_voidSendStringLine(u8 *A_Pu8String, LCD_ROWS row);

void HLCD_voidCreateCustomCharacters(void);
void HLCD_voidSendSpecialCharacter(u8 *A_pu8PatternArr, u8 A_u8PatternNumber);
```

```c
void HLCD_voidSendSpecialCharacter(u8 *A_pu8PatternArr, u8 A_u8PatternNumber)
{
    u8 local_u8CGRamAddress;

    // Calculate CGRAM Address = Pattern No. * 8
    local_u8CGRamAddress = A_u8PatternNumber * 8;

    // bit 6 must be high as it selects CGRAM to receive the next command
    SET_BIT(local_u8CGRamAddress, 6);

    // Send CGRAM Write Command
    HLCD_voidSendCommand(local_u8CGRamAddress);

    for(u8 i = 0; i < 7; i++)
    {
        HLCD_voidSendData(A_pu8PatternArr[i]);
    }

}
```

Custom LCD Characters

0 SOLID BLOCK

```
u8 solidBlock[7] = {
    0b01110,
    0b11111,
    0b11111,
    0b11111,
    0b11111,
    0b11111,
    0b01110
};
```

4 HEAD UP

```
u8 headUp[7] = {
    0b00000,
    0b00000,
    0b00000,
    0b00000,
    0b00100,
    0b01110,
    0b11111
};
```

3 HEAD LEFT

```
u8 headLeft[7] = {
    0b00001,
    0b00011,
    0b00111,
    0b01111,
    0b00111,
    0b00011,
    0b00001
};
```

2 HEAD RIGHT

```
u8 headRight[7] = {
    0b10000,
    0b11000,
    0b11100,
    0b11110,
    0b11100,
    0b11000,
    0b10000
};
```

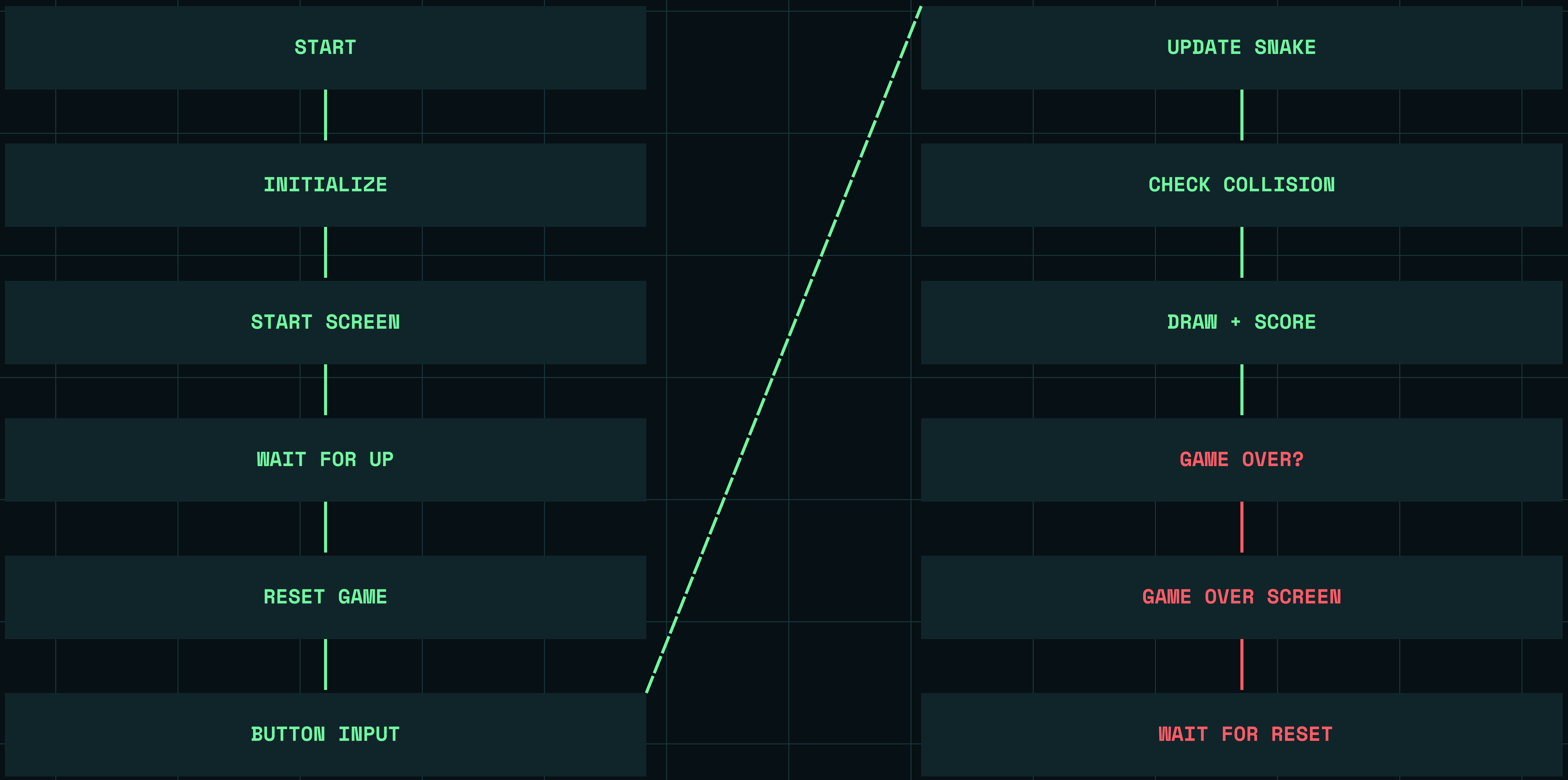
1 DIAMOND FOOD

```
u8 diamond[7] = {
    0b00000,
    0b00100,
    0b01110,
    0b11111,
    0b01110,
    0b00100,
    0b00000

};
```

5 HEAD DOWN

```
u8 headDown[7] = {
    0b11111,
    0b01110,
    0b00100,
    0b00000,
    0b00000,
    0b00000,
    0b00000

};
```

Game Loop Flowchart



MAIN

```c
int main(void)
{
    gameInit();
    while(1) {

        startScreen();
        gameReset();
        u8 g_gameOver= gameLoop();

        if(!g_gameOver) //got out of gameloop without losing,then RESET button is hit
        {
            continue;
        }

        gameOverScreen();

        while(1)
        {
            if(READ_RESET() == DIO_LOW)
            {
            _delay_ms(200);
            break;
            }
        }

    }
    return 0;
}
```

MAIN DATA TYPES USED

```c
static Snake g_snake; //struct holds all data related to snake
static Position g_food; //x-y coordinations of food

static u8 g_gameRunning; //flags
static u8 g_gameOver;

static u8 g_displayBuffer[4][20]; //all data to be sent to LCD

//initial value of button flags
static u8 g_prevUp = 1;
static u8 g_prevDown = 1;
static u8 g_prevLeft = 1;
static u8 g_prevRight = 1;
static u8 g_prevReset = 1;
```

```c
typedef enum
{
    MOVE_UP=0,
    MOVE_DOWN,
    MOVE_RIGHT,
    MOVE_LEFT,
    RESET
} BUTTON_PINS;

typedef struct {
    u8 x;
    u8 y;
} Position;

typedef enum {
    DIR_UP = 0,
    DIR_DOWN,
    DIR_LEFT,
    DIR_RIGHT,
    DIR_NONE
} Direction;

//SORTED BY SIZE TO AVOID MEMORY LEAK
typedef struct {
    Position body[MAX_SNAKE_LENGTH];
    u16 score;
    Direction nextDir;
    Direction currentDir;
    u8 length;
    u8 isDead;
} Snake;
```

snakeInit

```c
void snakeInit(void) //initializes data for each round
{
    //sets snake at the center of LCD ,directed to the right

    g_snake.length = 4;

    g_snake.currentDir = DIR_RIGHT;
    g_snake.nextDir = DIR_RIGHT;

    g_snake.isDead = 0;

    g_snake.score = 0;

    SevenSegment_DisplayNumber(0);

    g_snake.body[0].x = 8;
    g_snake.body[0].y = 1;

    g_snake.body[1].x = 7;
    g_snake.body[1].y = 1;

    g_snake.body[2].x = 6;
    g_snake.body[2].y = 1;

    g_snake.body[3].x = 5;
    g_snake.body[3].y = 1;


    HLCD_voidClearDisplay();

    generateFood();

    g_gameOver = 0;
    g_gameRunning = 1;

    g_prevUp = 1;
    g_prevDown = 1;
    g_prevLeft = 1;
    g_prevRight = 1;
    g_prevReset = 1;
}
```

Food functions

```c
void generateFood(void) //generate random coordinates of food position
{
    do {
        g_food.x = rand() % 20;// random number range => 0:19
        g_food.y = rand() % 4;// random number range => 0:3
    } while(checkFoodPosition(g_food.x, g_food.y));
}

u8 checkFoodPosition(u8 x, u8 y)// checks if the random position is on the snake body
{
    for(u8 i = 0; i < g_snake.length; i++)
    {
        if(g_snake.body[i].x == x && g_snake.body[i].y == y)
        {
            return 1;
        }
    }

    return 0;
}
```

button input

```c
#define READ_UP()     (MDIO_pinValueGetPinValue(BUTTON_PORT, MOVE_UP))
#define READ_DOWN()   (MDIO_pinValueGetPinValue(BUTTON_PORT, MOVE_DOWN))
#define READ_LEFT()   (MDIO_pinValueGetPinValue(BUTTON_PORT, MOVE_LEFT))
#define READ_RIGHT()  (MDIO_pinValueGetPinValue(BUTTON_PORT, MOVE_RIGHT))
#define READ_RESET()  (MDIO_pinValueGetPinValue(BUTTON_PORT, RESET))

#define IS_PRESSED(current, previous) ((current == DIO_LOW) && (previous == DIO_HIGH))
```

```c
void buttonInput(void) //handle input of buttons
{
    // checks falling edge

    u8 currentUp = READ_UP();
    u8 currentDown = READ_DOWN();
    u8 currentLeft = READ_LEFT();
    u8 currentRight = READ_RIGHT();
    u8 currentReset = READ_RESET();


    if(IS_PRESSED(currentReset, g_prevReset))
    {
        g_gameRunning=0;
        _delay_ms(200);
        g_prevReset = currentReset;

        return;
    }

    // prevent 180-direction change
    if(IS_PRESSED(currentUp, g_prevUp))
    {
        if(g_snake.currentDir != DIR_DOWN)
        {
            g_snake.nextDir = DIR_UP;
        }
    }
```

```c
    g_prevUp = currentUp;
    g_prevDown = currentDown;
    g_prevLeft = currentLeft;
    g_prevRight = currentRight;
    g_prevReset = currentReset;
}
```

snakeUpdate

```c
void snakeUpdate(void)// updates snake data
{
    if(g_snake.isDead || g_gameOver)
    {
        return;
    }


    g_snake.currentDir = g_snake.nextDir;


    Position newHead = g_snake.body[0]; //initialization


    switch(g_snake.currentDir)
    {
        case DIR_UP:
            newHead.y--;
            break;

        case DIR_DOWN:
            newHead.y++;
            break;

        case DIR_LEFT:
            newHead.x--;
            break;

        case DIR_RIGHT:
            newHead.x++;
            break;

        default:
            break;
    }
```

```c
    //checks if head at same position of food
    u8 foodEaten =(newHead.x == g_food.x && newHead.y == g_food.y);

    if(checkCollision(newHead))
    {
        g_snake.isDead = 1;
        g_gameOver = 1;
        return;
    }

    //shifts position data of body parts in the direction of the tail
    for(u8 i = g_snake.length - 1; i > 0; i--)
    {
        g_snake.body[i] = g_snake.body[i - 1];
    }

    g_snake.body[0] = newHead;

    if(foodEaten)
    {
        HBuzzer_voidBeep();

        if(g_snake.length < MAX_SNAKE_LENGTH)
        {
            g_snake.length++;

            g_snake.body[g_snake.length - 1] =
                g_snake.body[g_snake.length - 2];
        }

        g_snake.score += 5;
        SevenSegment_DisplayNumber((u8)g_snake.score);

        generateFood();
    }
}
```

snakeDraw

```c
void snakeDraw(void)
{
    //clears LCD quickly

    for(u8 row = 0; row < 4; row++)
    {
        for(u8 col = 0; col < 20; col++)
        {
            g_displayBuffer[row][col] = ' ';
        }
    }
```

```c
// assign position of snake to LCD buffer

for(u8 i = 0; i < g_snake.length; i++)
{
    u8 x = g_snake.body[i].x;
    u8 y = g_snake.body[i].y;
    u8 symbol;

    if(i == 0)
    {
        switch(g_snake.currentDir)
        {
             case DIR_UP:    symbol = HEAD_UP; break;
             case DIR_DOWN:  symbol = HEAD_DOWN; break;
             case DIR_LEFT:  symbol = HEAD_LEFT; break;
             case DIR_RIGHT: symbol = HEAD_RIGHT; break;
        }
            g_displayBuffer[y][x] = symbol;
    }
    else
    {
         g_displayBuffer[y][x] = SOLID_BLOCK;
    }
}

    g_displayBuffer[g_food.y][g_food.x] = DIAMOND;
```

```c
    //send data to LCD
for(u8 row = 0; row < 4; row++)
{
    HLCD_voidGoToPos(row + 1, col1);

    for(u8 col = 0; col < 20; col++)
    {
        HLCD_voidSendData(g_displayBuffer[row][col]);
    }
}

// Keep displaying the current score
SevenSegment_DisplayNumber((u8)g_snake.score);
```

Collision Detection Logic

```
u8 checkCollision(Position newHead)
{
    // check if snake is going to hit the borders

    if(newHead.x < 0 ||newHead.x >= 20 ||newHead.y < 0 ||newHead.y >= 4)
    {
        return 1;
    }

    // check if snake is going to hit its own body
    for(u8 i = 0; i < g_snake.length - 1; i++)
    {
        if(newHead.x == g_snake.body[i].x && newHead.y == g_snake.body[i].y)
        {
            return 1;
        }
    }

    return 0;
}
```

WALL COLLISION

SELF COLLISION

FOOD COLLISION

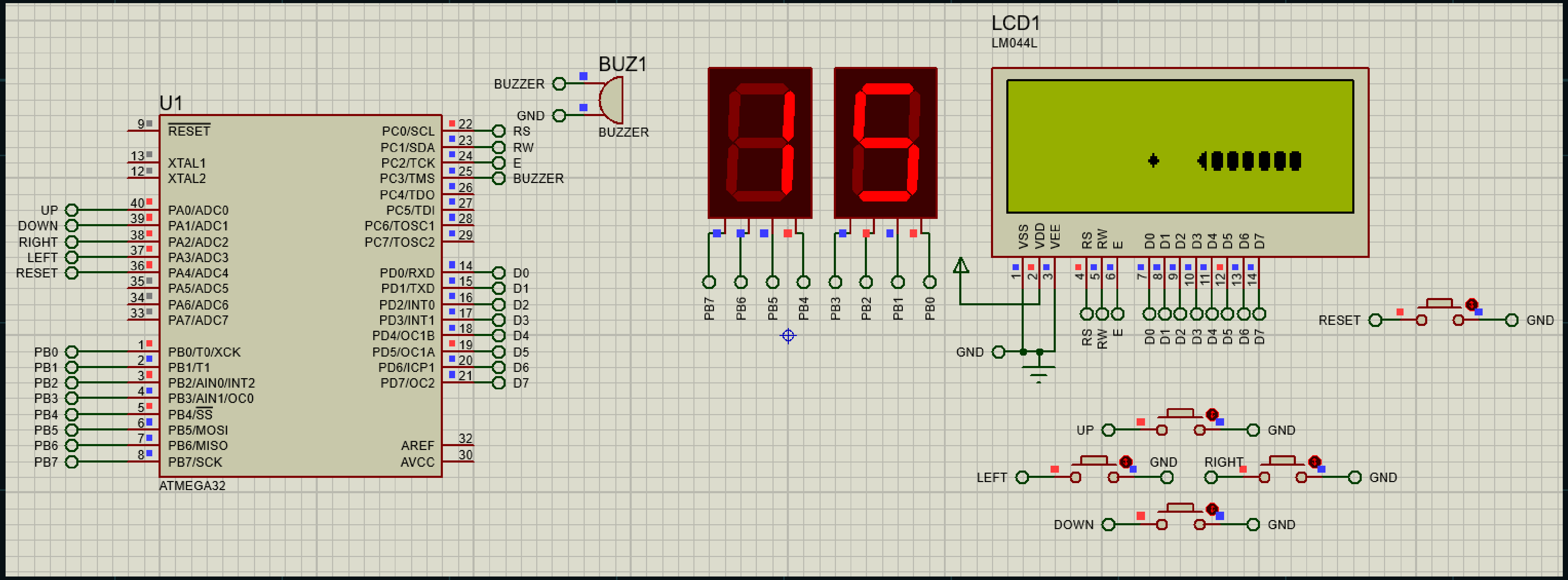
checkCollision(Position head)

if (head.x < 0 || head.x >= WIDTH || head.y < 0 || head.y >= HEIGHT) return true;
if (snakeBodyContains(head)) return true;
return false;

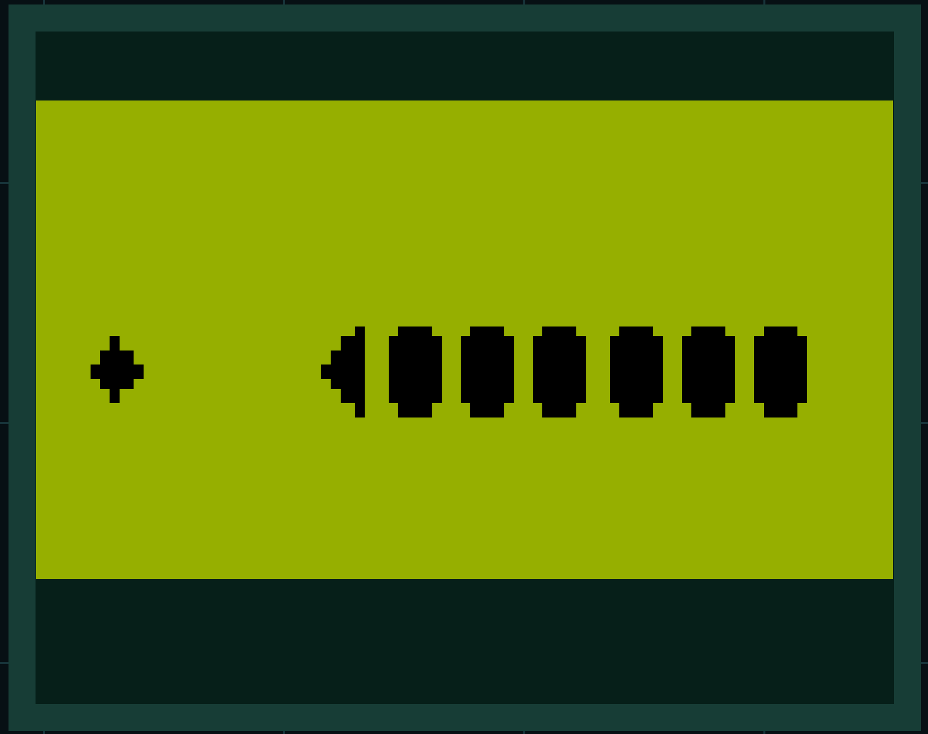
gameLoop

```c
u8 gameLoop(void)
{
    u8 frameCounter = 0;//counter to work with different speeds

    while(g_gameRunning) {
        buttonInput();//is checked every 50ms

        if(!g_gameRunning) {
            return 0;
        }

        if(frameCounter >= 5) {// frames are updated every 50*5=250ms
            snakeUpdate();
            frameCounter = 0;

            if(g_snake.isDead) {
                g_gameOver = 1;
                g_gameRunning = 0;

                return 1;
            }

            snakeDraw();
        }
        frameCounter++;
        _delay_ms(50);
    }
}
```

Game Experience



START SCREEN



GAMEPLAY

GAME OVER

Q&A

Questions / Discussion